

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МОЭВМ

ЛАБОРАТОРНАЯ РАБОТА №4
по дисциплине «Вычислительная математика»
Тема: интерполяция функций

Студент гр. 4341

Четвертная В.Л.

Преподаватель

Пуеров Г.Ю.

Санкт-Петербург

2026

Цель работы.

Изучить методы интерполяции функций (кубический сплайн и полином Лагранжа) и приобрести навыки их программной реализации для приближения функции.

Выполнения программы.

Программа написана на языке C++.

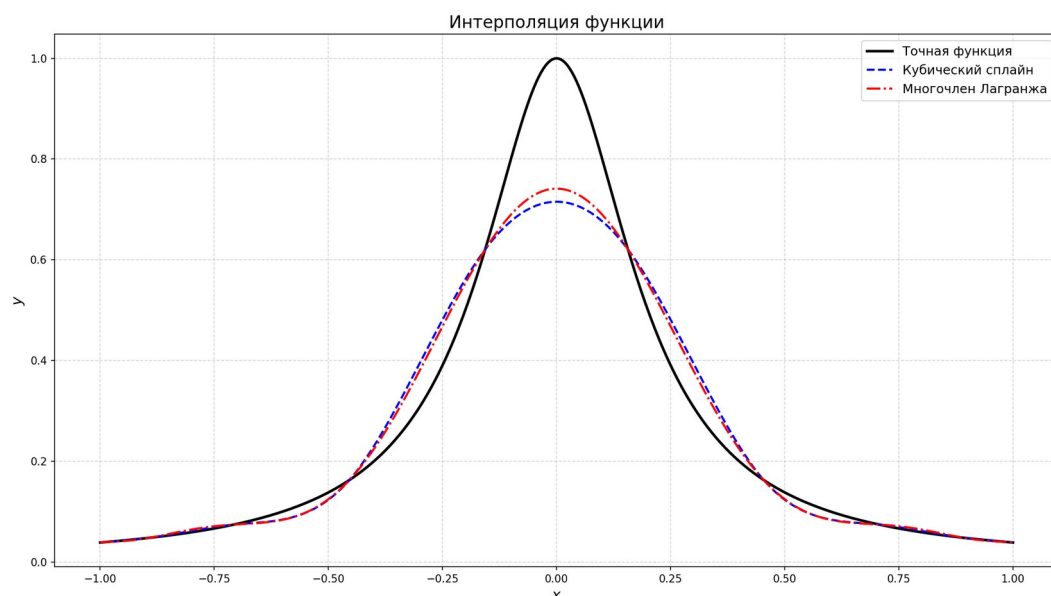
Содержит следующие ключевые функции:

- `double f(double x)` — вычисление функции Рунге.
- `void evaluate_lagrange(...)` — вычисление значений интерполяционного полинома Лагранжа в заданных точках по явной формуле.
- `void solve_tridiagonal(...)` — метод прогонки для решения трёхдиагональной системы.
- `void build_spline(...)` — построение кубического сплайна дефекта 1 с естественными граничными условиями (вторая производная на концах равна нулю, то есть $c[0] = c[n] = 0$). Находятся коэффициенты a_i , b_i , c_i , d_i для каждого отрезка $[x_i, x_{i+1}]$.
- `void evaluate_spline(...)` — вычисление значений сплайна в заданном наборе точек. При выходе точки за границы отрезка выполняется линейная экстраполяция по значению в ближайшем узле.

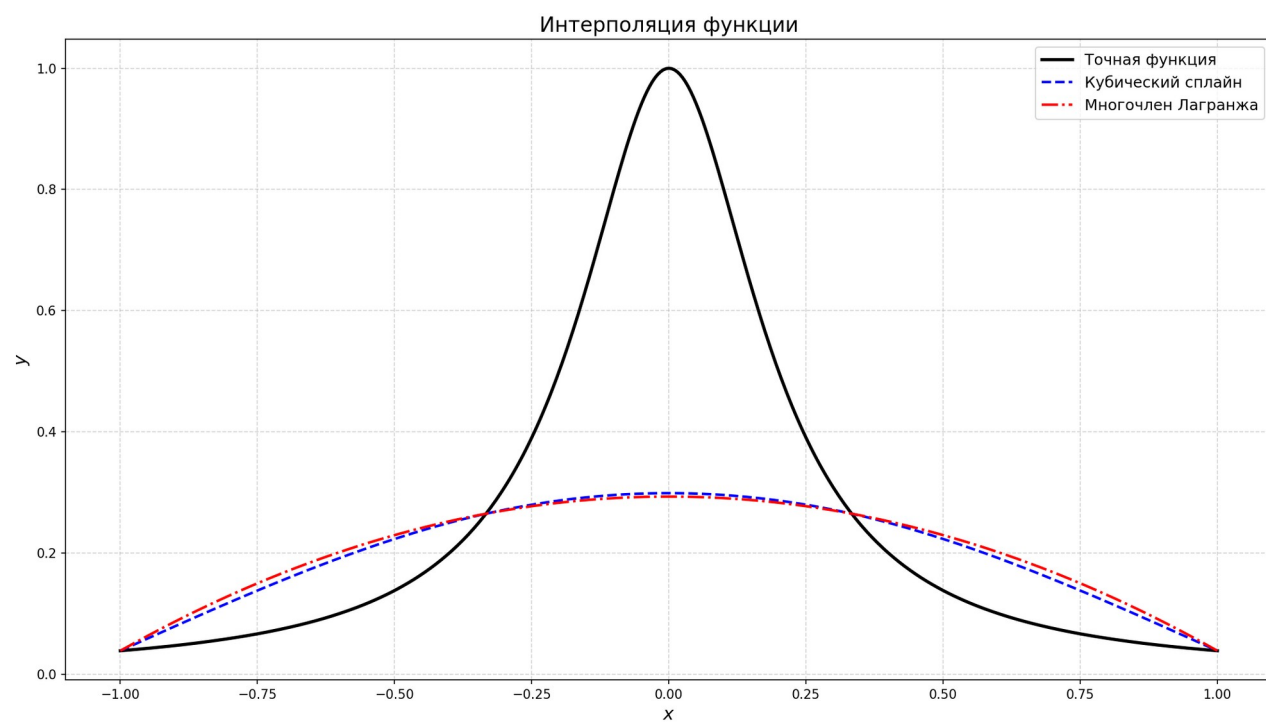
Код программы приведён в Приложении А.

Тестирование.

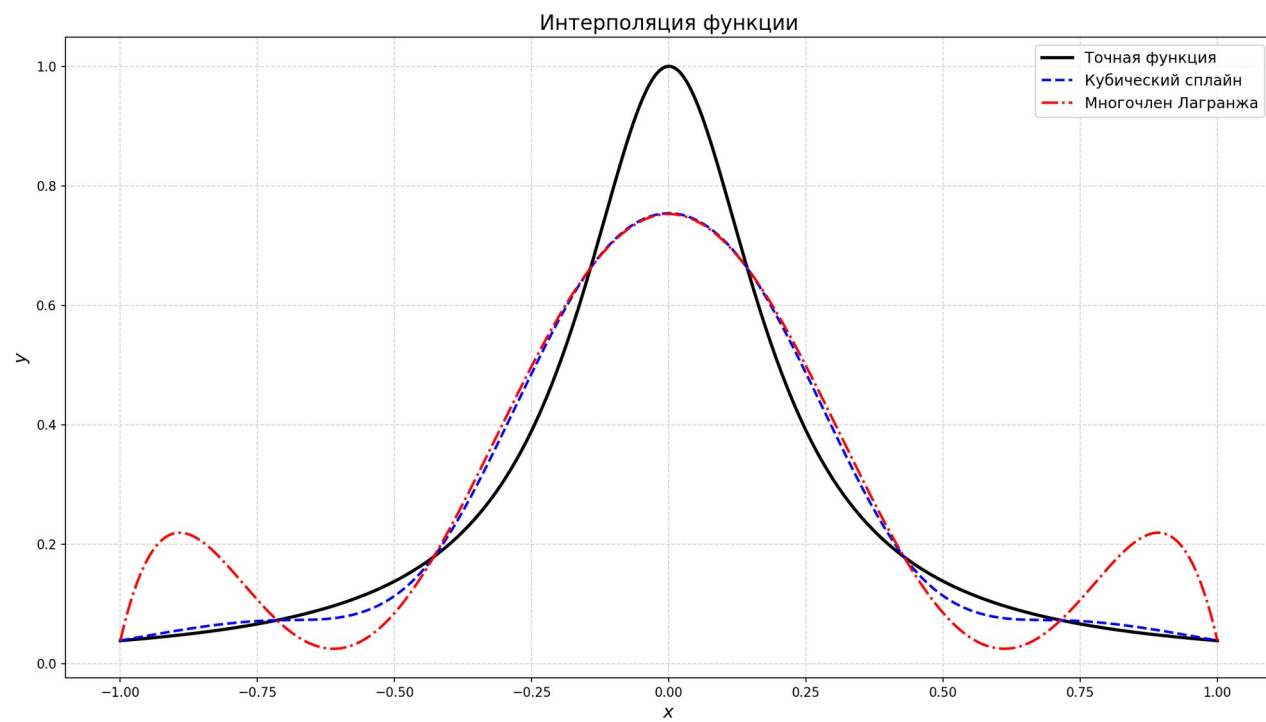
Узлы Чебышева



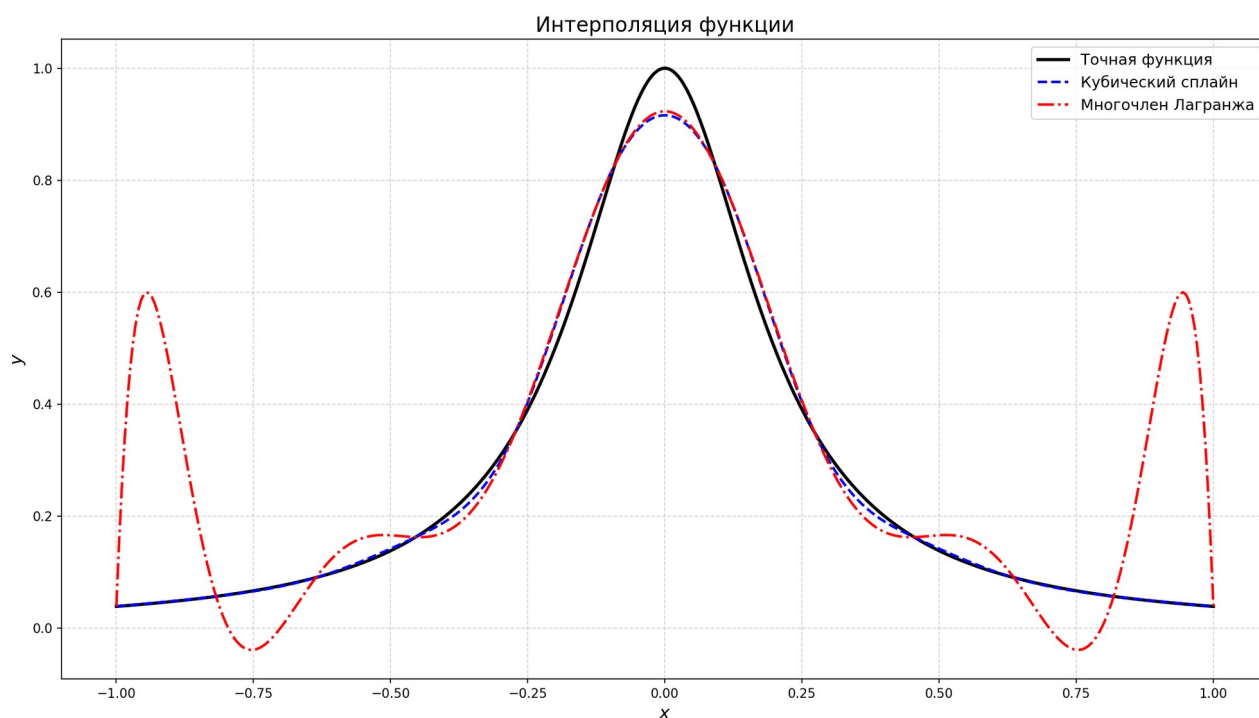
3 равномерных узла



7 равномерных узлов



11 равномерных узлов



Выводы.

Для Чебышевских узлов интерполяция многочленом Лагранжа и сплайном дают примерно одинаковые результаты, однако на равностоящих узлах видно преимущество интерполяции сплайнами — функция сходится плавно, без резких отклонений на концах отрезка.

ПРИЛОЖЕНИЕ А.

ИСХОДНЫЙ КОД ПРОГРАММЫ

```
#include <bits/stdc++.h>

const double PI=3.14159265358979323846;

double f(double x){
    return 1.0/(1.0+25*x*x);
}

void chebyshev_nodes(int n, double a, double b, std::vector
<double>& nodes){
    nodes[0] = a;
    nodes[n] = b;
    if (n > 1){
        for (int i=1; i < n; i++){
            double theta=(2.0*i-1.0)/(2.0*(n-1))*PI;
            nodes[i] = (a+b)/2.0+(b-a)/2.0*std::cos(theta);
        }
        std::sort(nodes.begin()+1, nodes.begin()+n);
    }
}

void evaluate_f(std::vector <double>& x, std::vector <double>& y){
    y.resize(x.size());
    for (size_t i=0; i < x.size(); i++)
        y[i] = f(x[i]);
}

void solve_tridiagonal(std::vector <double>& a, std::vector
<double>& b, std::vector <double>& c, std::vector <double>& d,
std::vector <double>& x){
    int n=d.size();
    x.resize(n);

    std::vector <double> p(n), q(n);
    p[0] = c[0]/b[0];
    q[0] = d[0]/b[0];

    for (int i=1; i < n; i++){
        double denom=b[i]-a[i]*p[i-1];

        p[i] = (i == n-1) ? 0.0 : c[i]/denom;
        q[i] = (d[i]-a[i]*q[i-1])/denom;
    }

    x[n-1] = q[n-1];
    for (int i=n-2; i >= 0; i--)
        x[i] = q[i]-p[i]*x[i+1];
}

void build_spline(std::vector <double>& x, std::vector <double>& y,
std::vector <double>& a, std::vector <double>& b, std::vector <double>&
c, std::vector <double>& d){
    int n=x.size()-1;
    std::vector <double> h(n);
```

```

    for (int i=0; i < n; i++)
        h[i] = x[i+1]-x[i];

    int N=n-1;
    std::vector <double> A(N, 0.0), B(N, 0.0), C(N, 0.0), RHS(N,
0.0);

    for (int i=0; i < N; i++){
        if (i == 0){
            A[i] = 0.0;
            B[i] = 2.0*(h[0]+h[1]);
            C[i] = h[1];
        }
        else if (i == N-1){
            A[i] = h[n-2];
            B[i] = 2.0*(h[n-2]+h[n-1]);
            C[i] = 0.0;
        }
        else{
            A[i] = h[i];
            B[i] = 2.0*(h[i]+h[i+1]);
            C[i] = h[i+1];
        }
        RHS[i] = 3.0*((y[i+2]-y[i+1])/h[i+1]-(y[i+1]-y[i])/h[i]);
    }

    std::vector <double> c_inner;
    solve_tridiagonal(A, B, C, RHS, c_inner);

    c.assign(n+1, 0.0);
    for (int i=0; i < N; i++)
        c[i+1] = c_inner[i];

    a.resize(n);
    b.resize(n);
    d.resize(n);
    for (int i=0; i < n; i++){
        a[i] = y[i];
        b[i] = (y[i+1]-y[i])/h[i]-h[i]*(2.0*c[i]+c[i+1])/3.0;
        d[i] = (c[i+1]-c[i])/(3.0*h[i]);
    }
}

void evaluate_spline(std::vector <double>& x_eval, std::vector
<double>& x_nodes, std::vector <double>& a, std::vector <double>& b,
std::vector <double>& c, std::vector <double>& d, std::vector <double>&
results){
    results.resize(x_eval.size());
    int n=x_nodes.size()-1;

    for (size_t k=0; k < x_eval.size(); k++){
        double t=x_eval[k];
        if (t <= x_nodes[0])
            t = x_nodes[0];
        if (t >= x_nodes[n])
            t = x_nodes[n];

```

```

        auto it=std::upper_bound(x_nodes.begin(), x_nodes.end(),
t);
        int i=std::distance(x_nodes.begin(), it)-1;
        if (i < 0)
            i = 0;
        if (i >= n)
            i = n-1;

        double dx=t-x_nodes[i];
        results[k] = a[i]+b[i]*dx+c[i]*dx*dx+d[i]*dx*dx*dx;
    }
}

void evaluate_lagrange(std::vector <double>& x_eval, std::vector
<double>& x_nodes, std::vector <double>& y_nodes, std::vector <double>&
results){
    results.resize(x_eval.size());
    int m=x_nodes.size();

    for (size_t k=0; k < x_eval.size(); ++k){
        double xx=x_eval[k], sum=0.0;

        for (int i=0; i < m; i++){
            double term=y_nodes[i];
            for (int j=0; j < m; ++j){
                if (j != i)
                    term *= (xx-x_nodes[j])/(x_nodes[i]-
x_nodes[j]);
            }
            sum += term;
        }
        results[k] = sum;
    }
}

int main(){
    double a=-1.0, b=1.0;
    int n, plot_points=400;
    std::cin >> n;

    std::vector <double> nodes(n+1);
    // chebyshev_nodes(n, a, b, nodes);
    nodes[0] = a;
    nodes[n] = b;
    double h=(b-a)/n;
    for (int i=1; i < n; i++)
        nodes[i] = a+i*h;

    std::vector <double> y_nodes;
    evaluate_f(nodes, y_nodes);

    std::vector <double> A, B, C, D;
    build_spline(nodes, y_nodes, A, B, C, D);

    std::vector <double> x_plot(plot_points+1);
    for (int k=0; k <= plot_points; k++)
        x_plot[k] = a+(b-a)*k/plot_points;

```

```

        std::vector <double> f_vals, spline_vals, lagrange_vals;
        evaluate_f(x_plot, f_vals);
        evaluate_spline(x_plot, nodes, A, B, C, D, spline_vals);
        evaluate_lagrange(x_plot, nodes, y_nodes, lagrange_vals);

        std::ofstream out("plot_data.txt");
        for (size_t i=0; i < x_plot.size(); i++)
            out << x_plot[i] << " " << f_vals[i] << " " <<
spline_vals[i] << " " << lagrange_vals[i] << "\n";

        out.close();
    }

import matplotlib.pyplot as plt
import numpy as np

file = open('plot_data.txt', 'r')
x, f_vals, S_vals, L_vals = [], [], [], []

for line in file:
    a, b, c, d = map(float, line.split())
    x.append(a)
    f_vals.append(b)
    S_vals.append(c)
    L_vals.append(d)

file.close()

x_array = np.array(x)
f_array = np.array(f_vals)
S_array = np.array(S_vals)
L_array = np.array(L_vals)

plt.figure(figsize=(14, 8))

plt.plot(x_array, f_array, 'k-', linewidth=2.5, label='Точная
функция')
plt.plot(x_array, S_array, 'b--', linewidth=2, label='Кубический
сплайн')
plt.plot(x_array, L_array, 'r-.', linewidth=2, label='Многочлен
Лагранжа')

plt.xlabel('$x$', fontsize=14, fontweight='bold')
plt.ylabel('$y$', fontsize=14, fontweight='bold')
plt.title('Интерполяция функции', fontsize=16)
plt.legend(loc='best', fontsize=12)

plt.grid(True, alpha=0.5, linestyle='--')

plt.tight_layout()
plt.savefig('interpolation_plot.png', dpi=150)

```